

APUNTES DEFINITIVOS FRONTEND - EXAMEN IISSI2

Estos apuntes cubren el flujo completo que te pedirán en el examen: Listar datos (GET) -> Crear/Editar con formulario (POST/PUT) -> Borrar pidiendo confirmación (DELETE).

EL CEREBRO DEL COMPONENTE: useState y useEffect

Estos dos "hooks" son obligatorios en cualquier pantalla que muestre datos de la base de datos.

- useState (La Memoria): Sirve para guardar variables que, si cambian, obligan a la pantalla a actualizarse y repintarse.

Uso típico: Guardar la lista de datos que te devuelve la API (restaurantes, pedidos, direcciones) o controlar si un modal está abierto (true) o cerrado (false).

Ejemplo: `const [addresses, setAddresses] = useState([])`

- useEffect (El Ejecutor): Sirve para ejecutar código automáticamente sin que el usuario pulse nada.

Uso típico: Llamar a tu función que hace el GET al backend nada más entrar a la pantalla.

Regla de oro: SIEMPRE ponle un array de dependencias al final []. Si no lo pones, hará peticiones infinitas al backend y suspenderás por colapsar el servidor.

Ejemplo de código:

```
const [addresses, setAddresses] = useState([])
```

```
useEffect(() => {  
  fetchAddresses() // Se ejecuta solo una vez al cargar la pantalla  
}, []) // <- ¡El array vacío es vital!
```

MOSTRAR LISTAS: FlatList y funciones render

En todos los exámenes te piden listar algo. NUNCA hagas un "map" manual si es una lista principal; usa siempre el componente .

Parámetros clave del FlatList:

- data: Le pasas tu variable del useState (ej. addresses).

- `keyExtractor`: Le dices cómo identificar cada elemento de forma única (siempre suele ser: `item => item.id.toString()`).
- `renderItem`: Aquí llamas a la función que dibuja cada fila (ej. `renderAddress`).

¿Cómo se hace la función `render`?

Se extrae el "item" entre llaves y se devuelve un componente visual. Aquí es donde pones los textos y los botones de Editar o Borrar.

Ejemplo de código:

```
const renderAddress = ({ item }) => (

  {item.alias}: {item.street}, {item.city}

  {/* Botón de Borrar que abre el Modal */}
  <Pressable onPress={() => {
    setElementoSeleccionado(item)
    setModalVisible(true)
  }}>
    <MaterialCommunityIcons name="delete" size={24} color="red" />
  </Pressable>
  // En el return principal de tu pantalla:
  <FlatList
    data={addresses}
    renderItem={renderAddress}
    keyExtractor={item => item.id.toString()}
  />
```

LAYOUT Y ESTILOS: View y StyleSheet

React Native no usa CSS tradicional, usa un objeto JavaScript con `StyleSheet.create`.

- `View`: Es el equivalente a un `div` en HTML. Sirve para agrupar elementos.
- `Flexbox`: Es la herramienta para alinear.

Por defecto, React Native apila las cosas en columna (de arriba a abajo).

Si quieres poner un texto a la izquierda y unos botones a la derecha (como en la lista de direcciones), necesitas poner los elementos en fila.

Ejemplo de código:

```
const styles = StyleSheet.create({
  row: {
    flexDirection: 'row', // Pone los elementos en fila horizontal
    justifyContent: 'space-between', // Los separa empujándolos a los bordes
    padding: 15,
    borderBottomWidth: 1,
    borderColor: '#ccc'
  }
});
```

```
}  
})
```

FORMULARIOS: Formik y Yup (POST y PUT)

Esta es la parte donde más puntos te juegas (ej. crear un horario o editar una dirección). Formik gestiona los datos que el usuario escribe y Yup valida que sean correctos antes de enviarlos.

Pasos clave:

initialValues: Si estás creando (POST), pásale cadenas vacías ". Si estás editando (PUT), pásale los valores que ya vienen del backend.

validationSchema: Aquí aplicas reglas (obligatorio, mayor que 0, formato email, etc.).

onSubmit: La función que llama al backend y navega a la pantalla anterior si todo va bien.

Ejemplo de código:

```
import * as yup from 'yup'  
import { Formik } from 'formik'
```

// 1. Crear las reglas de validación

```
const validationSchema = yup.object().shape({  
  address: yup.string().required('La dirección es obligatoria'),  
  price: yup.number().positive('Debe ser mayor que 0').required('Requerido')  
})
```

// 2. Dentro del return de tu pantalla:

```
<Formik  
  initialValues={{ address: order.address || "", price: order.price || "" }}  
  validationSchema={validationSchema}  
  onSubmit={async (values) => {  
    try {  
      await updateOrder(order.id, values) // Tu llamada a la API (Endpoint)  
      navigation.goBack() // Volver a la lista tras actualizar correctamente  
    } catch (error) {  
      showMessage({ message: 'Error al guardar', type: 'error' })  
    }  
  }}  
>
```

```
{{{ handleChange, handleBlur, handleSubmit, values, errors }}} => (
```

```
<TextInput  
  onChangeText={handleChange('address')}  
  onBlur={handleBlur('address')}  
  value={values.address}  
>
```

```
{errors.address && {errors.address}}
```

```
<Button onPress={handleSubmit} title="Save" />
</View>
})
```

PETICIONES DELETE (El Modal de Confirmación)

Si te piden borrar algo, SIEMPRE te van a pedir que muestres un aviso de confirmación antes de borrarlo definitivamente de la base de datos.

Pasos clave:

Crea un estado para ver el modal: `const [modalVisible, setModalVisible] = useState(false)`

Crea un estado para saber qué ID borrar: `const [idToDelete, setIdToDelete] = useState(null)`

Al pulsar el icono de la papelera en tu lista, actualizas esos estados:

```
setIdToDelete(item.id)
setModalVisible(true)
```

En el botón de "Confirmar" dentro del Modal, llamas a la API, cierras el modal y recargas la lista.

Ejemplo de código de la función de borrado:

```
const handleDelete = async () => {
  try {
    await deleteEndpoint(idToDelete) // Llamada DELETE a la API
    setModalVisible(false) // Cierras el modal
    fetchDatos() // Vuelves a hacer el GET para que la lista se repinte sin el elemento borrado
    showMessage({ message: 'Borrado con éxito', type: 'success' })
  } catch (error) {
    showMessage({ message: 'Error al borrar', type: 'error' })
  }
}
```

RESUMEN DE SUPERVIVENCIA PARA EL EXAMEN:

• FLUJO MENTAL:

- ¿Tengo que pintar datos en pantalla? -> `useEffect` + `useState` + `FlatList`.
- ¿Tengo que crear o editar datos? -> `Formik` + `Yup` + API (`POST/PUT`).
- ¿Tengo que borrar un dato? -> Estado Modal + API (`DELETE`) + Recargar lista.

• ERRORES TONTOS A EVITAR:

- Olvidar poner el `"await"` antes de hacer una llamada a la API.
- Olvidar exportar la función nueva que crees en el archivo de Endpoints.
- Olvidar poner el array vacío `[]` al final del `useEffect`.

